

Real-Time Examples of Abstract Class in C#

In this article, I will discuss **Multiple Real-Time Examples of Abstract Classes in C#**. Please read our [Abstract Class in C#](#) article, where we discussed the basic concepts of Abstract Class. At the end of this article, you will understand the following Real-time Examples using the Abstract Class in C#.

1. [What is an Abstract Class in C#?](#)
2. [Banking System](#)
3. [Shapes Management](#)
4. [Animal Kingdom](#)
5. [Vehicles Management System](#)
6. [Employees in an Organization](#)
7. [Media Players](#)
8. [Advantages and Disadvantages of Abstract Class in C#](#)
9. [When to use Abstract Class in C#?](#)

What is an Abstract Class in C#?

An abstract class in C# is a special class that cannot be instantiated directly. Instead, it serves as a base class for other classes. An abstract class can contain abstract methods (which don't have any implementation) and non-abstract methods (which have implementations). Derived classes inheriting from an abstract class must provide implementations for its abstract methods. Here are the key characteristics and functionalities of an abstract class in C#:

- **Cannot be Instantiated:** You can't create an object of an abstract class using the new keyword.
- **Abstract Methods:** An abstract class can contain abstract methods which don't have a body. Derived classes must override and provide an implementation for these methods.
- **Non-Abstract Methods:** An abstract class can contain non-abstract methods with a defined body. Derived classes can use these methods as they are or override them if required.
- **Properties and Fields:** Abstract classes can contain properties and fields like any other class.
- **Constructors:** While it may seem counter-intuitive, abstract classes can have constructors. Though you can't instantiate an abstract class directly, the derived classes can call the constructor of the abstract class.
- **Access Modifiers:** Abstract methods in an abstract class can have access modifiers like public, protected, etc. If no access modifier is specified, protected is the default.

Real-Time Example of Abstract Class in C#: Banking System

Let's see a real-time example of an abstract class in C# by considering a scenario related to the Banking System.

- **Abstract Class – BankAccount**
- **Concrete Classes – SavingsAccount, CurrentAccount**

Let us implement this example using Abstract Class in C#:

```
using System;  
namespace AbstractClassCSharp
```

```

{
    //BankAccount.cs (Abstract Class)
    public abstract class BankAccount
    {
        public string AccountNumber { get; private set; }
        public string AccountHolder { get; private set; }
        protected double Balance { get; set; }
        public BankAccount(string accountNumber, string accountHolder)
        {
            AccountNumber = accountNumber;
            AccountHolder = accountHolder;
        }
        // Abstract methods
        public abstract void Deposit(double amount);
        public abstract void Withdraw(double amount);
        // Virtual method with a default implementation
        public virtual void DisplayBalance()
        {
            Console.WriteLine($"Account Number: {AccountNumber}, Account Holder: {AccountHolder}, Balance: ${Balance}");
        }
    }
    //SavingsAccount.cs (Concrete Class)
    public class SavingsAccount : BankAccount
    {
        private double _interestRate = 0.03; // 3% annual interest
        public SavingsAccount(string accountNumber, string accountHolder)
            : base(accountNumber, accountHolder) { }
        public override void Deposit(double amount)
        {
            Balance += amount;
            Console.WriteLine($"Deposited ${amount} into Savings Account.");
        }
        public override void Withdraw(double amount)
        {
            if (Balance - amount >= 0)
            {
                Balance -= amount;
                Console.WriteLine($"Withdrew ${amount} from Savings Account.");
            }
            else
            {
                Console.WriteLine("Insufficient funds for withdrawal.");
            }
        }
        public void AddInterest()
        {
            Balance += Balance * _interestRate;
            Console.WriteLine($"Interest added. New Balance: ${Balance}");
        }
    }
}

```

```
//CurrentAccount.cs (Concrete Class)
public class CurrentAccount : BankAccount
{
    private double _overdraftLimit = 1000.0;
    public CurrentAccount(string accountNumber, string accountHolder)
        : base(accountNumber, accountHolder) { }
    public override void Deposit(double amount)
    {
        Balance += amount;
        Console.WriteLine($"Deposited ${amount} into Current Account.");
    }
    public override void Withdraw(double amount)
    {
        if ((Balance - amount) >= -_overdraftLimit)
        {
            Balance -= amount;
            Console.WriteLine($"Withdrew ${amount} from Current Account.");
        }
        else
        {
            Console.WriteLine("Withdrawal exceeds overdraft limit.");
        }
    }
}

//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        SavingsAccount johnsSavings = new SavingsAccount("SA123456", "John");
        johnsSavings.Deposit(1000);
        johnsSavings.Withdraw(200);
        johnsSavings.AddInterest();
        johnsSavings.DisplayBalance();
        CurrentAccount janesCurrent = new CurrentAccount("CA654321", "Jane");
        janesCurrent.Deposit(500);
        janesCurrent.Withdraw(1000);
        janesCurrent.DisplayBalance();
        Console.ReadKey();
    }
}
}
```

In this example, the abstract class BankAccount defines any bank account's essential structure and functionalities, such as depositing and withdrawing money. The concrete classes, SavingsAccount and CurrentAccount, provide specific implementations for these functionalities and may have additional features like adding interest or handling overdrafts. When you run the above code, you will get the following output:

```
Deposited $1000 into Savings Account.  
Withdrew $200 from Savings Account.  
Interest added. New Balance: $824  
Account Number: SA123456, Account Holder: John Doe, Balance: $824  
Deposited $500 into Current Account.  
Withdrew $1000 from Current Account.  
Account Number: CA654321, Account Holder: Jane Smith, Balance: $-500
```

Real-Time Example of Abstract Class in C#: Shapes

Let's use the classic example of a shape hierarchy to illustrate the concept of an abstract class in C#.

- **Abstract Class – Shape**
- **Concrete Classes – Circle, Rectangle**

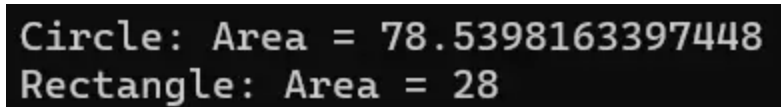
Let us implement this example using Abstract Class in C#:

```
using System;  
using System.Collections.Generic;  
namespace AbstractClassCSharp  
{  
    //Shape.cs (Abstract Class)  
    public abstract class Shape  
    {  
        public string Name { get; set; }  
        // Abstract method with no body  
        public abstract double Area();  
        // Virtual method with a default implementation  
        public virtual void Display()  
        {  
            Console.WriteLine($"{Name}: Area = {Area()}");  
        }  
    }  
    //Circle.cs (Concrete Class)  
    public class Circle : Shape  
    {  
        public double Radius { get; set; }  
        public Circle(double radius)  
        {  
            Name = "Circle";  
            Radius = radius;  
        }  
        // Concrete implementation of the Area method for Circle  
        public override double Area()  
        {  
            return Math.PI * Radius * Radius;  
        }  
    }  
}
```

```
//Rectangle.cs (Concrete Class)
public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public Rectangle(double width, double height)
    {
        Name = "Rectangle";
        Width = width;
        Height = height;
    }
    // Concrete implementation of the Area method for Rectangle
    public override double Area()
    {
        return Width * Height;
    }
}

//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        Circle circle = new Circle(5);
        Rectangle rectangle = new Rectangle(4, 7);
        circle.Display();
        rectangle.Display();
        Console.ReadKey();
    }
}
```

When you run the main program, the output will display the areas for the circle and the rectangle, as shown in the image below. Notice how the abstract class Shape forces each derived class to implement the Area method, ensuring that every shape we create must have a way to calculate its area. The Display method, on the other hand, provides a default implementation but can be overridden in derived classes if necessary.



```
Circle: Area = 78.5398163397448
Rectangle: Area = 28
```

Real-Time Example of Abstract Class in C#: Animal Kingdom

Let's see another real-time example of an abstract class in C# by considering a scenario related to the animal kingdom.

- **Abstract Class – Animal**
- **Concrete Classes – Dog, Fish**

Let us implement this example using Abstract Class in C#:

```

using System;
namespace AbstractClassCSharp
{
    //Animal.cs (Abstract Class)
    public abstract class Animal
    {
        public string Name { get; set; }
        // Abstract method with no body
        public abstract void Speak();
        // Virtual method with a default implementation
        public virtual void Eat()
        {
            Console.WriteLine($"{Name} is eating.");
        }
    }
    //Dog.cs (Concrete Class)
    public class Dog : Animal
    {
        public Dog(string name)
        {
            Name = name;
        }
        // Concrete implementation of the Speak method for Dog
        public override void Speak()
        {
            Console.WriteLine($"{Name} says: Woof!");
        }
    }
    //Fish.cs (Concrete Class)
    public class Fish : Animal
    {
        public Fish(string name)
        {
            Name = name;
        }
        // Fish don't traditionally "speak", so we'll implement this different
        public override void Speak()
        {
            Console.WriteLine($"{Name} bubbles!");
        }
        // Overriding the Eat method specifically for Fish
        public override void Eat()
        {
            Console.WriteLine($"{Name} is nibbling on some seaweed.");
        }
    }

    //Step 4: Testing the application.
    class Program
    {
        static void Main(string[] args)

```

```

    {
        Dog dog = new Dog("Buddy");
        Fish fish = new Fish("Nemo");
        dog.Speak();
        dog.Eat();
        fish.Speak();
        fish.Eat();

        Console.ReadKey();
    }
}

```

When you run the main program, you will observe the different behaviors of the Dog and Fish classes. The abstract Animal class ensures that all animal types must have a way to Speak, but how they do so (barking, bubbling, etc.) depends on the specific animal. The Eat method provides a default behavior but can also be overridden for specific animals, like in the case of the Fish.

Real-Time Example of Abstract Class in C#: Vehicles

Let's see another real-time example of an abstract class in C# by considering a scenario related to the Vehicles.

- **Abstract Class – Vehicle**
- **Concrete Classes – Car, Bicycle**

Let us implement this example using Abstract Class in C#:

```

using System;
namespace AbstractClassCSharp
{
    //Vehicle.cs (Abstract Class)
    public abstract class Vehicle
    {
        public string Brand { get; set; }
        // Abstract method with no body
        public abstract void Move();
        // Virtual method with a default implementation
        public virtual void Refuel()
        {
            Console.WriteLine($"{Brand} is refueling.");
        }
    }
    //Car.cs (Concrete Class)
    public class Car : Vehicle
    {
        public Car(string brand)
        {
            Brand = brand;
        }
    }
}

```

```

// Concrete implementation of the Move method for Car
public override void Move()
{
    Console.WriteLine($"{Brand} car is driving.");
}
// Overriding the Refuel method for Car
public override void Refuel()
{
    Console.WriteLine($"{Brand} car is filling up with gas.");
}
}
//Bicycle.cs (Concrete Class)
public class Bicycle : Vehicle
{
    public Bicycle(string brand)
    {
        Brand = brand;
    }
    // Concrete implementation of the Move method for Bicycle
    public override void Move()
    {
        Console.WriteLine($"{Brand} bicycle is pedaling.");
    }
    // Overriding the Refuel method specifically for Bicycle as they don't
traditionally refuel
    public override void Refuel()
    {
        Console.WriteLine($"{Brand} bicycle doesn't need to refuel.");
    }
}

//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        Car toyota = new Car("Toyota");
        Bicycle trek = new Bicycle("Trek");
        toyota.Move();
        toyota.Refuel();
        trek.Move();
        trek.Refuel();
        Console.ReadKey();
    }
}
}

```

When you run the main program, you will notice the distinct behaviors of the Car and Bicycle classes. The abstract Vehicle class mandates that all vehicle types have a method to Move. However, the exact behavior of that movement (driving or pedaling) depends on the specific type of vehicle. The Refuel method offers a

default behavior but can be customized for specific vehicle types, as seen with the car and the bicycle in this example.

Real-Time Example of Abstract Class in C#: Employees in an Organization

Let's see another real-time example of an abstract class in C# by considering a scenario related to the Employees in an organization.

- **Abstract Class – Employee**
- **Concrete Classes – Manager, Developer**

Let us implement this example using Abstract Class in C#:

```
using System;
namespace AbstractClassCSharp
{
    //Employee.cs (Abstract Class)
    public abstract class Employee
    {
        public string Name { get; set; }
        public string EmployeeID { get; set; }
        // Constructor
        public Employee(string name, string employeeID)
        {
            Name = name;
            EmployeeID = employeeID;
        }
        // Abstract method with no body
        public abstract void PerformTask();
        // Virtual method with a default implementation
        public virtual void AttendMeeting()
        {
            Console.WriteLine($"{Name} is attending a general meeting.");
        }
    }
    //Manager.cs (Concrete Class)
    public class Manager : Employee
    {
        public Manager(string name, string employeeID) : base(name, employeeID)
        // Concrete implementation of the PerformTask method for Manager
        public override void PerformTask()
        {
            Console.WriteLine($"{Name} is assigning tasks to team members.");
        }
        // Overriding the AttendMeeting method for Manager
        public override void AttendMeeting()
        {
            Console.WriteLine($"{Name} is attending a managerial meeting.");
        }
    }
}
```

```

//Developer.cs (Concrete Class)
public class Developer : Employee
{
    public Developer(string name, string employeeID) : base(name, employeeID)
    {
        // Concrete implementation of the PerformTask method for Developer
        public override void PerformTask()
        {
            Console.WriteLine($"{Name} is writing code.");
        }
    }
}

//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        Manager alice = new Manager("Alice", "M001");
        Developer bob = new Developer("Bob", "D001");
        alice.PerformTask();
        alice.AttendMeeting();
        bob.PerformTask();
        bob.AttendMeeting();
        Console.ReadKey();
    }
}

```

In this example, the abstract class `Employee` provides a basic structure for various types of employees in an organization. Each specific type of employee, such as a `Manager` or `Developer`, has tasks and activities unique to their role. Using an abstract class ensures that all derived employee types implement the `PerformTask` method. Additionally, a virtual method, `AttendMeeting`, has a default behavior but can be overridden for specific employee roles if necessary.

Real-Time Example of Abstract Class in C#: Media Players

Let's see another real-time example of an abstract class in C# by considering a scenario related to the Media Players.

- **Abstract Class – MediaPlayer**
- **Concrete Classes – AudioPlayer, VideoPlayer**

Let us implement this example using Abstract Class in C#:

```

using System;
namespace AbstractClassCSharp
{
    //MediaPlayer.cs (Abstract Class)
    public abstract class MediaPlayer
    {
        public string FileName { get; set; }
    }
}

```

```

// Abstract methods with no body
public abstract void Play();
public abstract void Pause();
public abstract void Stop();
// Virtual method with a default implementation
public virtual void LoadFile(string fileName)
{
    FileName = fileName;
    Console.WriteLine($"Loaded file: {FileName}");
}
}
//AudioPlayer.cs (Concrete Class)
public class AudioPlayer : MediaPlayer
{
    // Concrete implementations of the abstract methods
    public override void Play()
    {
        Console.WriteLine($"Playing audio: {FileName}");
    }
    public override void Pause()
    {
        Console.WriteLine($"Pausing audio: {FileName}");
    }
    public override void Stop()
    {
        Console.WriteLine($"Stopping audio: {FileName}");
    }
}
//VideoPlayer.cs (Concrete Class)
public class VideoPlayer : MediaPlayer
{
    // Concrete implementations of the abstract methods
    public override void Play()
    {
        Console.WriteLine($"Playing video: {FileName}");
    }
    public override void Pause()
    {
        Console.WriteLine($"Pausing video: {FileName}");
    }
    public override void Stop()
    {
        Console.WriteLine($"Stopping video: {FileName}");
    }
}

//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {

```

```

        AudioPlayer music = new AudioPlayer();
        music.LoadFile("song.mp3");
        music.Play();
        music.Pause();
        music.Stop();
        VideoPlayer movie = new VideoPlayer();
        movie.LoadFile("movie.mp4");
        movie.Play();
        movie.Pause();
        movie.Stop();
        Console.ReadKey();
    }
}

```

In this example, the abstract class `MediaPlayer` outlines the basic functions any media player should have (Play, Pause, Stop) but does not define how these functions will work—that's left for the concrete classes. The concrete classes `AudioPlayer` and `VideoPlayer` then provide specific implementations for these functionalities.

This abstraction allows developers to create more media player types in the future, maybe for streaming or other media formats, ensuring they all adhere to a standard set of operations.

Advantages and Disadvantages of Abstract Class in C#

Abstract classes in C# are midway between base classes (ordinary classes) and interfaces. They come with their own set of advantages and disadvantages:

Advantages of Abstract Classes in C#:

- **Code Reusability:** Abstract classes allow you to define default implementations for methods (unlike interfaces in C# versions before 8.0). This promotes reusability since derived classes can utilize the default implementation and only override what's necessary.
- **Flexibility:** Abstract classes can define a mix of methods. They can have abstract methods (without implementations) and concrete methods (with implementations). This provides a balance of enforcing a contract and providing a default behavior.
- **State Maintenance:** Unlike interfaces, abstract classes can have fields and properties, allowing them to maintain state.
- **Constructor Logic:** Abstract classes can have constructors. This can be helpful to set up initial states or enforce certain conditions when a derived class is instantiated.
- **Control:** Using abstract classes, you can enforce a hierarchy or structure for the derived classes. For instance, you can control the access levels of methods, properties, or fields to ensure that they are used correctly by the derived classes.

Disadvantages of Abstract Classes in C#:

- **Single Inheritance:** One of the most significant drawbacks of abstract classes is that C# does not support multiple inheritance for classes. A class cannot inherit from more than one class (whether abstract or not). So, if a class already inherits from an abstract class, it can't inherit from another.

- **Increased Complexity:** For larger or improperly designed systems, having a web of abstract and concrete-derived classes can make the system harder to understand, maintain, and debug.
- **Versioning Issues:** If you need to add a new method to an abstract class in a library others are using, you might break their existing derived classes. Those classes would now be missing implementations for the new abstract methods.
- **Less Flexibility in Derived Classes:** Derived classes are bound to the structure and behavior defined in the abstract class. Sometimes, this can be too restrictive if a derived class needs to exhibit behavior that diverges significantly from the defined contract.
- **Overhead:** There can be overhead defining abstract methods that might be overridden in all derived classes. This might make interfaces a better option in such scenarios.

When to use Abstract Class in C#?

In C#, deciding when to use an abstract class typically depends on the specific requirements of your design. Here are some scenarios or criteria to help determine when it might be appropriate to use an abstract class:

- **Default Implementation:** An abstract class is ideal if you want to provide a default implementation for some methods but still want to force subclasses to implement other methods. The abstract class can contain abstract (no implementation) and concrete (with implementation) methods.
- **Shared State:** When multiple derived classes should share some common state (fields or properties), then an abstract class is beneficial. Unlike interfaces, abstract classes can have fields and properties, allowing you to encapsulate state.
- **Controlled Inheritance:** If you want to control the inheritance chain, i.e., ensure that a particular piece of code is always executed when a method in a derived class is called, you can use an abstract class and define that code in the base abstract class.
- **Shared Base Logic with Constructors:** Abstract classes can have constructors, which can be useful to set up some initial state or enforce certain initial conditions for the derived classes.
- **Versioning:** If you're building a library and there's a chance you'll add more methods in the future, an abstract class offers a more straightforward path for versioning. Adding new methods to an interface would break existing implementations, but adding them to an abstract class (with a default implementation) would not.
- **Is-A Relationship:** If the derived classes should inherently be a type of base class, then an abstract class is suitable. For instance, in a scenario with a base class Bird and derived classes Eagle and Penguin, an "Eagle is a Bird" relationship makes sense, and an abstract class can be used to define common bird attributes and behaviors.
- **Single Inheritance:** If you're sure that the derived class doesn't need to inherit from multiple sources, then an abstract class is a valid choice. C# doesn't support multiple inheritance for classes, so once a class inherits from an abstract class, it cannot inherit from another class.

However, there are certain situations where interfaces might be a more appropriate choice:

- **Multiple Inheritances of Type:** If you want a class to adopt multiple contracts or behaviors without specifying the implementation, then interfaces are your best bet. A class can implement multiple interfaces.
- **Pure Contract Definition:** If you only want to define a contract that multiple classes should adhere to without any default implementation, then an interface makes sense (though C# 8.0 and onwards allow default implementations in interfaces).

Whether to use an abstract class or not depends on the specific requirements of your design. If you want to enforce a specific contract but also provide some default behaviors or maintain a state, abstract classes can be beneficial. However, if you want to ensure a contract, interfaces might be a better choice, especially with the enhancements to interfaces in C# 8.0 and later, which allow default interface implementations.

In the next article, I will discuss the **Multiple Real-time Examples of Static Classes in C#**. I explain **Real-Time Examples of Abstract Classes** in this article in C#. I hope you enjoy this Abstract Class Real-Time Examples using the C# article.